

NAME: S.Devi Shree

REGISTER NO: 192511149

COURSE NAME: Database Management Systems

COURSE CODE: CSA0503

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 35%

Code of Conduct:

I S.Devi Shree(192511149) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

1.Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.

SELECT

S.StudentName,

T.TeacherName AS ClassTeacher,

S.Grade

FROM Student S

INNER JOIN Class C

ON S.ClassID = C.ClassID

INNER JOIN Teacher T

ON C.TeacherID = T.TeacherID;

```
mysql> desc Teacher;
```

Field	Type	Null	Key	Default	Extra
TeacherID	int	NO	PRI	NULL	
TeacherName	varchar(50)	YES		NULL	
Subject	varchar(30)	YES		NULL	

3 rows in set (0.05 sec)

```
mysql> desc Class;
```

Field	Type	Null	Key	Default	Extra
ClassID	int	NO	PRI	NULL	
ClassName	varchar(20)	YES		NULL	
TeacherID	int	YES	MUL	NULL	

3 rows in set (0.01 sec)

```
mysql> desc Student;
```

Field	Type	Null	Key	Default	Extra
StudentID	int	NO	PRI	NULL	
StudentName	varchar(50)	YES		NULL	
Grade	char(2)	YES		NULL	
ClassID	int	YES	MUL	NULL	

4 rows in set (0.01 sec)

```
mysql> select * from Teacher;
```

TeacherID	TeacherName	Subject
1	Mr. Kumar	Mathematics
2	Mrs. Meena	Science
3	Ms. Lakshmi	English

3 rows in set (0.01 sec)

```
mysql> select * from Class;
```

ClassID	ClassName	TeacherID
101	Class X-A	1
102	Class X-B	2
103	Class X-C	3

3 rows in set (0.00 sec)

```
mysql> select * from Student;
```

StudentID	StudentName	Grade	ClassID
1001	Ravi	A	101
1002	Priya	A+	102
1003	Karan	B+	101
1004	Anu	A	103

4 rows in set (0.00 sec)

```
mysql> SELECT S.StudentID, S.StudentName,T.TeacherName AS ClassTeacher,S.Grade FROM Student S INNER JOIN Class C ON S.ClassID = C.ClassID INNER JOIN Teacher T ON C.TeacherID = T.TeacherID;
```

StudentID	StudentName	ClassTeacher	Grade
1001	Ravi	Mr. Kumar	A
1003	Karan	Mr. Kumar	B+
1002	Priya	Mrs. Meena	A+
1004	Anu	Ms. Lakshmi	A

```
4 rows in set (0.01 sec)
```

2.A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.

```
SELECT
    D.DoctorID,
    D.DoctorName,
    COUNT(A.AppointmentID) AS TotalAppointments
FROM Doctor D
INNER JOIN Appointment A
ON D.DoctorID = A.DoctorID
WHERE MONTH(A.AppointmentDate) = MONTH(CURRENT_DATE)
    AND YEAR(A.AppointmentDate) = YEAR(CURRENT_DATE)
GROUP BY D.DoctorID, D.DoctorName
HAVING COUNT(A.AppointmentID) > 10;
```

```
mysql> CREATE TABLE Patient (PatientID INT PRIMARY KEY,PatientName VARCHAR(50),Age INT,Gender VARCHAR(10));
Query OK, 0 rows affected (0.28 sec)
```

```
mysql> desc Patient;
```

Field	Type	Null	Key	Default	Extra
PatientID	int	NO	PRI	NULL	
PatientName	varchar(50)	YES		NULL	
Age	int	YES		NULL	
Gender	varchar(10)	YES		NULL	

```
4 rows in set (0.09 sec)
```

```
mysql> CREATE TABLE Doctor (DoctorID INT PRIMARY KEY,DoctorName
VARCHAR(50),Specialization VARCHAR(50));
Query OK, 0 rows affected (0.07 sec)
```

```
mysql> desc Doctor;
```

Field	Type	Null	Key	Default	Extra
DoctorID	int	NO	PRI	NULL	
DoctorName	varchar(50)	YES		NULL	
Specialization	varchar(50)	YES		NULL	

3 rows in set (0.17 sec)

```
mysql> CREATE TABLE Appointment (AppointmentID INT PRIMARY KEY,P
atientID INT,DoctorID INT,AppointmentDate DATE,AppointmentTime T
IME,FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),FOREIG
N KEY (DoctorID) REFERENCES Doctor(DoctorID));
Query OK, 0 rows affected (0.34 sec)
```

```
mysql> desc Appointment;
```

Field	Type	Null	Key	Default	Extra
AppointmentID	int	NO	PRI	NULL	
PatientID	int	YES	MUL	NULL	
DoctorID	int	YES	MUL	NULL	
AppointmentDate	date	YES		NULL	
AppointmentTime	time	YES		NULL	

5 rows in set (0.01 sec)

```
mysql> INSERT INTO Patient VALUES(101,'Ravi',25,'Male'),(102,'Pr
iya',30,'Female'),(103,'Karan',40,'Male'),(104,'Anu',35,'Female'
),(105,'Rahul',28,'Male');
Query OK, 5 rows affected (0.05 sec)
Records: 5 Duplicates: 0 Warnings: 0
```

```
mysql> select * from Patient;
```

PatientID	PatientName	Age	Gender
101	Ravi	25	Male
102	Priya	30	Female
103	Karan	40	Male
104	Anu	35	Female
105	Rahul	28	Male

5 rows in set (0.01 sec)

```
mysql> select * from Doctor;
```

DoctorID	DoctorName	Specialization
1	Dr. Kumar	Cardiologist
2	Dr. Meena	Neurologist
3	Dr. Arjun	Orthopedic

3 rows in set (0.01 sec)

```
mysql> select * from Appointment;
```

AppointmentID	PatientID	DoctorID	AppointmentDate	AppointmentTime
1001	101	1	2026-08-01	09:00:00
1002	102	1	2026-08-02	10:00:00
1003	103	1	2026-08-03	11:00:00
1004	104	1	2026-08-04	12:00:00
1005	105	1	2026-08-05	09:30:00
1006	101	1	2026-08-06	10:30:00
1007	102	1	2026-08-07	11:30:00
1008	103	1	2026-08-08	09:15:00
1009	104	1	2026-08-09	10:45:00
1010	105	1	2026-08-10	11:15:00
1011	101	1	2026-08-11	12:15:00
1012	102	2	2026-08-12	09:45:00
1013	103	2	2026-08-13	10:15:00
1014	104	3	2026-08-14	11:45:00

```
14 rows in set (0.01 sec)
```

```
mysql> SELECT
->     D.DoctorID,
->     D.DoctorName,
->     COUNT(A.AppointmentID) AS TotalAppointments
-> FROM Doctor D
-> INNER JOIN Appointment A
-> ON D.DoctorID = A.DoctorID
-> WHERE MONTH(A.AppointmentDate) = MONTH(CURRENT_DATE)
-> AND YEAR(A.AppointmentDate) = YEAR(CURRENT_DATE)
-> GROUP BY D.DoctorID, D.DoctorName
-> HAVING COUNT(A.AppointmentID) > 10;
```

DoctorID	DoctorName	TotalAppointments
1	Dr. Kumar	11

```
1 row in set (0.05 sec)
```

3. In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.

Create Product Table `CREATE TABLE Product ( Product_ID INT PRIMARY KEY, Product_Name VARCHAR(50), Price DECIMAL(10,2) );`

Create Orders Table `CREATE TABLE Orders ( Order_ID INT PRIMARY KEY, Product_ID INT, Quantity INT, Order_Date DATE, FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID) );`

Insert into Product `INSERT INTO Product VALUES (101,'Laptop',50000), (102,'Mobile',25000), (103,'Headphones',3000), (104,'Keyboard',1500), (105,'Mouse',800);`

Insert into Orders `INSERT INTO Orders VALUES (1,101,2,'2026-08-01'), (2,102,1,'2026-08-02'), (3,103,5,'2026-08-03');`

Display Tables `SELECT * FROM Product; SELECT * FROM Orders;`

Subquery `SELECT Product_ID, Product_Name, Price FROM Product WHERE Product_ID NOT IN ( SELECT Product_ID FROM Orders );`

```
mysql> SELECT * FROM Product;
```

Product_ID	Product_Name	Price
101	Laptop	50000.00
102	Mobile	25000.00
103	Headphones	3000.00
104	Keyboard	1500.00
105	Mouse	800.00

```
5 rows in set (0.02 sec)
```

```
mysql> SELECT * FROM Orders;
```

Order_ID	Product_ID	Quantity	Order_Date
1	101	2	2026-08-01
2	102	1	2026-08-02
3	103	5	2026-08-03

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Product WHERE Product_ID > 100;
```

Product_ID	Product_Name	Price
104	Keyboard	1500.00
105	Mouse	800.00

```
2 rows in set (0.01 sec)
```

4. For a university database, write relational algebra expressions to find students enrolled in both DBMS and OS courses.

Assume the Relations

STUDENT(SID, StudentName)

COURSE(CID, CourseName)

ENROLLMENT(SID, CID)

Step 1: Find students enrolled in DBMS

DBMS_Students $\leftarrow \pi_{SID}$

$(\sigma_{CourseName='DBMS'}(COURSE) \bowtie ENROLLMENT)$

Step 2: Find students enrolled in OS

OS_Students $\leftarrow \pi_{SID}$

$(\sigma_{CourseName='OS'}(COURSE) \bowtie ENROLLMENT)$

Step 3: Find students enrolled in BOTH courses

$DBMS_Students \cap OS_Students$

Step 4: Display Student Names

$\pi_{StudentName}$

$(STUDENT \bowtie (DBMS_Students \cap OS_Students))$

Final Relational Algebra Expression

$\pi_{StudentName}$

$(STUDENT \bowtie (\pi_{SID}(\sigma_{CourseName='DBMS'}(COURSE) \bowtie ENROLLMENT) \cap$

$\pi_{SID}(\sigma_{CourseName='OS'}(COURSE) \bowtie ENROLLMENT)))$

5. Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.

CREATE TABLE Department (

Dept_ID VARCHAR(5) PRIMARY KEY,

Dept_Name VARCHAR(50)

);

CREATE TABLE Employee (

Emp_ID INT PRIMARY KEY,

Emp_Name VARCHAR(50),

Salary DECIMAL(10,2),

Dept_ID VARCHAR(5),

FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)

);

INSERT INTO Department

VALUES

('D1','HR'),

('D2','Finance'),

('D3','IT');

INSERT INTO Employee

VALUES

(101,'Ravi',40000,'D1'),

```

(102,'Priya',55000,'D1'),
(103,'Karan',45000,'D2'),
(104,'Anu',70000,'D2'),
(105,'John',50000,'D3'),
(106,'Meena',80000,'D3');
SELECT * FROM Employee;
CREATE VIEW BelowDeptAverage AS
SELECT
    E.Emp_ID,
    E.Emp_Name,
    E.Salary,
    E.Dept_ID
FROM Employee E
WHERE E.Salary <
(
    SELECT AVG(Salary)
    FROM Employee
    WHERE Dept_ID = E.Dept_ID
);
SELECT * FROM BelowDeptAverage;
UPDATE Employee
SET Salary = Salary * 1.10
WHERE Salary <
(
    SELECT AvgSalary
    FROM
    (
        SELECT Dept_ID,
            AVG(Salary) AS AvgSalary
        FROM Employee
        GROUP BY Dept_ID
    )

```

) AS DeptAvg

WHERE DeptAvg.Dept_ID = Employee.Dept_ID

);

SELECT * FROM Employee;

```
mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+
| Emp_ID | Emp_Name | Salary | Dept_ID |
+-----+-----+-----+-----+
| 101 | Ravi | 40000.00 | D1 |
| 102 | Priya | 55000.00 | D1 |
| 103 | Karan | 45000.00 | D2 |
| 104 | Anu | 70000.00 | D2 |
| 105 | John | 50000.00 | D3 |
| 106 | Meena | 80000.00 | D3 |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM BelowDeptAverage;
+-----+-----+-----+-----+
| Emp_ID | Emp_Name | Salary | Dept_ID |
+-----+-----+-----+-----+
| 101 | Ravi | 40000.00 | D1 |
| 103 | Karan | 45000.00 | D2 |
| 105 | John | 50000.00 | D3 |
+-----+-----+-----+-----+
3 rows in set (0.01 sec)
```

```
mysql> -- Display Updated Employee Table
mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+
| Emp_ID | Emp_Name | Salary | Dept_ID |
+-----+-----+-----+-----+
| 101 | Ravi | 44000.00 | D1 |
| 102 | Priya | 55000.00 | D1 |
| 103 | Karan | 49500.00 | D2 |
| 104 | Anu | 70000.00 | D2 |
| 105 | John | 55000.00 | D3 |
| 106 | Meena | 80000.00 | D3 |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

6. In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.

CREATE TABLE Customer (

Customer_ID INT PRIMARY KEY,

Customer_Name VARCHAR(50),

Phone VARCHAR(15)

);

```

CREATE TABLE Branch (
    Branch_ID INT PRIMARY KEY,
    Branch_Name VARCHAR(50),
    City VARCHAR(30)
);

CREATE TABLE Account (
    Account_No INT PRIMARY KEY,
    Customer_ID INT,
    Branch_ID INT,
    Account_Type VARCHAR(20),
    Balance DECIMAL(10,2),
    FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
    FOREIGN KEY (Branch_ID) REFERENCES Branch(Branch_ID)
);

CREATE TABLE Transactions (
    Transaction_ID INT PRIMARY KEY,
    Account_No INT,
    Transaction_Date DATE,
    Transaction_Type VARCHAR(20),
    Amount DECIMAL(10,2),
    FOREIGN KEY (Account_No) REFERENCES Account(Account_No)
);

INSERT INTO Customer
VALUES
(101,'Ravi','9876543210'),
(102,'Priya','9876501234'),
(103,'Karan','9123456789');

INSERT INTO Branch
VALUES
(1,'Chennai Main','Chennai'),
(2,'Coimbatore Branch','Coimbatore'),

```

(3,'Madurai Branch','Madurai');

INSERT INTO Transactions

VALUES

(1,10001,'2026-08-01','Deposit',10000),

(2,10001,'2026-08-03','Withdrawal',5000),

(3,10002,'2026-08-02','Deposit',15000),

(4,10003,'2026-08-04','Withdrawal',3000);

SELECT * FROM Customer;

```
mysql> SELECT * FROM Customer;
+-----+-----+-----+
| Customer_ID | Customer_Name | Phone |
+-----+-----+-----+
|          101 | Ravi          | 9876543210 |
|          102 | Priya         | 9876501234 |
|          103 | Karan         | 9123456789 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

SELECT * FROM Branch;

```
mysql> SELECT * FROM Branch;
+-----+-----+-----+
| Branch_ID | Branch_Name      | City |
+-----+-----+-----+
|          1 | Chennai Main     | Chennai |
|          2 | Coimbatore Branch | Coimbatore |
|          3 | Madurai Branch   | Madurai |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

SELECT * FROM Account;

```
mysql> SELECT * FROM Account;
+-----+-----+-----+-----+-----+
| Account_No | Customer_ID | Branch_ID | Account_Type | Balance |
+-----+-----+-----+-----+-----+
|          10001 |          101 |          1 | Savings      | 50000.00 |
|          10002 |          102 |          2 | Current      | 75000.00 |
|          10003 |          103 |          3 | Savings      | 30000.00 |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

SELECT * FROM Transactions;

```
mysql> SELECT * FROM Transactions;
+-----+-----+-----+-----+-----+
| Transaction_ID | Account_No | Transaction_Date | Transaction_Type | Amount |
+-----+-----+-----+-----+-----+
|          1 |          10001 | 2026-08-01 | Deposit | 10000.00 |
|          2 |          10001 | 2026-08-03 | Withdrawal | 5000.00 |
|          3 |          10002 | 2026-08-02 | Deposit | 15000.00 |
|          4 |          10003 | 2026-08-04 | Withdrawal | 3000.00 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```

SELECT

    C.Customer_ID,

    C.Customer_Name,

    A.Account_No,

    A.Account_Type,

    A.Balance,

    T.Transaction_Date,

    T.Transaction_Type,

    T.Amount,

    B.Branch_Name,

    B.City

FROM Customer C

INNER JOIN Account A

ON C.Customer_ID = A.Customer_ID

INNER JOIN Transactions T

ON A.Account_No = T.Account_No

INNER JOIN Branch B

ON A.Branch_ID = B.Branch_ID;

```

```
--> ON A.Branch_ID = B.Branch_ID;
```

Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Transaction_Date	Transaction_Type	Amount	Branch_Name
City								
101	Ravi	10001	Savings	50000.00	2026-08-01	Deposit	10000.00	Chennai Main
Chennai								
101	Ravi	10001	Savings	50000.00	2026-08-03	Withdrawal	5000.00	Chennai Main
Chennai								
102	Priya	10002	Current	75000.00	2026-08-02	Deposit	15000.00	Coimbatore Br
anch								
103	Karan	10003	Savings	30000.00	2026-08-04	Withdrawal	3000.00	Madurai Branc
h								
Madurai								

```

4 rows in set (0.01 sec)

```

7.A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs.10,000.

```

CREATE TABLE Sales (

    Sale_ID INT PRIMARY KEY,

    Product_Name VARCHAR(50),

    Category VARCHAR(30),

    Quantity INT,

```

```

Unit_Price DECIMAL(10,2),

Sale_Date DATE

);

INSERT INTO Sales

VALUES

(1,'Laptop','Electronics',2,50000,'2026-08-01'),

(2,'Mouse','Electronics',5,800,'2026-08-01'),

(3,'Notebook','Stationery',20,100,'2026-08-01'),

(4,'Pen','Stationery',30,50,'2026-08-01'),

(5,'Office Chair','Furniture',3,6000,'2026-08-01'),

(6,'Table','Furniture',2,8000,'2026-08-01'),

(7,'Monitor','Electronics',3,15000,'2026-08-01');

SELECT * FROM Sales;

```

```

mysql> SELECT * FROM Sales;
+-----+-----+-----+-----+-----+-----+
| Sale_ID | Product_Name | Category | Quantity | Unit_Price | Sale_Date |
+-----+-----+-----+-----+-----+-----+
| 1 | Laptop | Electronics | 2 | 50000.00 | 2026-08-01 |
| 2 | Mouse | Electronics | 5 | 800.00 | 2026-08-01 |
| 3 | Notebook | Stationery | 20 | 100.00 | 2026-08-01 |
| 4 | Pen | Stationery | 30 | 50.00 | 2026-08-01 |
| 5 | Office Chair | Furniture | 3 | 6000.00 | 2026-08-01 |
| 6 | Table | Furniture | 2 | 8000.00 | 2026-08-01 |
| 7 | Monitor | Electronics | 3 | 15000.00 | 2026-08-01 |
+-----+-----+-----+-----+-----+-----+
7 rows in set (0.02 sec)

```

```

SELECT

Category,

SUM(Quantity * Unit_Price) AS Total_Sales

FROM Sales

GROUP BY Category

HAVING SUM(Quantity * Unit_Price) > 10000;

```

```

mysql> SELECT
+-----+-----+
| Category | Total_Sales |
+-----+-----+
| Electronics | 149000.00 |
| Furniture | 34000.00 |
+-----+-----+
2 rows in set (0.04 sec)

```

8. For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.

Assume the Relations

MEMBER

MEMBER(Member_ID, Member_Name)

BOOK

BOOK(Book_ID, Book_Title, Author)

BORROW

BORROW(Member_ID, Book_ID, Borrow_Date)

Tuple Relational Calculus (TRC) Expression

{M.Member_Name |

$M \in \text{MEMBER} \wedge \exists B \in \text{BOOK}$

$(B.\text{Author} = \text{'C.J. Date'} \wedge \exists BR \in \text{BORROW} (BR.\text{Member_ID} = M.\text{Member_ID} \wedge BR.\text{Book_ID} = B.\text{Book_ID}))\}$

9. Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.

CREATE TABLE Supplier (

Supplier_ID INT PRIMARY KEY,

Supplier_Name VARCHAR(50),

City VARCHAR(30)

);

CREATE TABLE Product (

Product_ID INT PRIMARY KEY,

Product_Name VARCHAR(50),

Price DECIMAL(10,2) CHECK (Price > 0),

Stock INT CHECK (Stock >= 0),

Supplier_ID INT,

FOREIGN KEY (Supplier_ID)

REFERENCES Supplier(Supplier_ID)

);

INSERT INTO Supplier

VALUES

(1,'ABC Suppliers','Chennai'),

(2,'XYZ Traders','Coimbatore'),

(3,'Global Distributors','Madurai');

INSERT INTO Product

VALUES

(101,'Laptop',50000,20,1),

(102,'Mouse',800,100,2),

(103,'Keyboard',1500,50,2),

(104,'Monitor',12000,15,3);

SELECT * FROM Supplier;

```
mysql> SELECT * FROM Supplier;
+-----+-----+-----+
| Supplier_ID | Supplier_Name | City |
+-----+-----+-----+
| 1 | ABC Suppliers | Chennai |
| 2 | XYZ Traders | Coimbatore |
| 3 | Global Distributors | Madurai |
+-----+-----+-----+
3 rows in set (0.01 sec)
```

SELECT * FROM Product;

```
mysql>
mysql> SELECT * FROM Product;
+-----+-----+-----+-----+-----+
| Product_ID | Product_Name | Price | Stock | Supplier_ID |
+-----+-----+-----+-----+-----+
| 101 | Laptop | 50000.00 | 20 | 1 |
| 102 | Mouse | 800.00 | 100 | 2 |
| 103 | Keyboard | 1500.00 | 50 | 2 |
| 104 | Monitor | 12000.00 | 15 | 3 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Product;
+-----+-----+-----+-----+-----+
| Product_ID | Product_Name | Price | Stock | Supplier_ID |
+-----+-----+-----+-----+-----+
| 101 | Laptop | 50000.00 | 20 | 1 |
| 102 | Mouse | 800.00 | 100 | 2 |
| 103 | Keyboard | 1500.00 | 50 | 2 |
| 104 | Monitor | 12000.00 | 15 | 3 |
| 108 | Speaker | 3000.00 | 25 | 1 |
+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

10. Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.

```
CREATE TABLE Department (  
    Dept_ID INT PRIMARY KEY,  
    Dept_Name VARCHAR(50)  
);  
  
CREATE TABLE Employee (  
    Emp_ID INT PRIMARY KEY,  
    Emp_Name VARCHAR(50),  
    Salary DECIMAL(10,2),  
    Dept_ID INT,  
    FOREIGN KEY (Dept_ID)  
    REFERENCES Department(Dept_ID)  
);  
  
INSERT INTO Department  
VALUES  
(1,'HR'),  
(2,'Finance'),  
(3,'IT'),  
(4,'Marketing');  
  
INSERT INTO Employee  
VALUES  
(101,'Ravi',50000,1),  
(102,'Priya',65000,2),  
(103,'Karan',45000,1),  
(104,'Anu',70000,3),  
(105,'John',55000,2),
```

```

(106,'Meena',80000,3),
(107,'Rahul',60000,4);

SELECT * FROM Department;

SELECT * FROM Employee;

CREATE VIEW HR_DepartmentSummary AS

SELECT

    D.Dept_Name,

    COUNT(E.Emp_ID) AS Total_Employees,

    AVG(E.Salary) AS Average_Salary

FROM Department D

INNER JOIN Employee E

ON D.Dept_ID = E.Dept_ID

GROUP BY D.Dept_Name;

SELECT * FROM HR_DepartmentSummary;

SELECT * FROM HR_DepartmentSummary

WHERE Average_Salary > 50000;

```

```

mysql> SELECT * FROM Department;
+-----+-----+
| DeptID | DeptName |
+-----+-----+
| 1      | HR       |
| 2      | Finance  |
| 3      | IT       |
| 4      | Marketing|
| D1     | HR       |
| D2     | Finance  |
| D3     | IT       |
+-----+-----+
7 rows in set (0.01 sec)

```

```

mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+-----+
| Emp_ID | Emp_Name | Salary | Dept_ID | Project_ID |
+-----+-----+-----+-----+-----+
| 101    | Ravi     | 50000.00 | D1      | D1         |
| 102    | Priya    | 60000.00 | D2      | D2         |
| 103    | Karan    | 45000.00 | D1      | D3         |
| 104    | Anu      | 70000.00 | D3      | NULL       |
| 105    | John     | 55000.00 | D2      | NULL       |
| 106    | Meena    | 40000.00 | D3      | NULL       |
+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)

```

```
mysql> SELECT * FROM HR_DepartmentSummary;
```

DeptName	Total_Employees	Average_Salary
HR	2	47500.000000
Finance	2	57500.000000
IT	2	55000.000000

3 rows in set (0.03 sec)

```
-> WHERE Average_Salary > 50000;
```

DeptName	Total_Employees	Average_Salary
Finance	2	57500.000000
IT	2	55000.000000

2 rows in set (0.01 sec)